#Hashlock.

5ire

(Staking)

SMART CONTRACT

Security Audit

Performed on Contracts:

IFireChainToken.sol

MD5 Hash:f027ad9dff1ddc4dc50f36b9ee9a0ef1

proxy.sol

MD5 Hash:f1dba7f90d696316074cc1cf3bd4634c

Staking2.sol

MD5 Hash 4f59cee5686f54ec71e418dbef5b8f31:

Platform

ETH

hashlock.com.au

MAY 2024

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

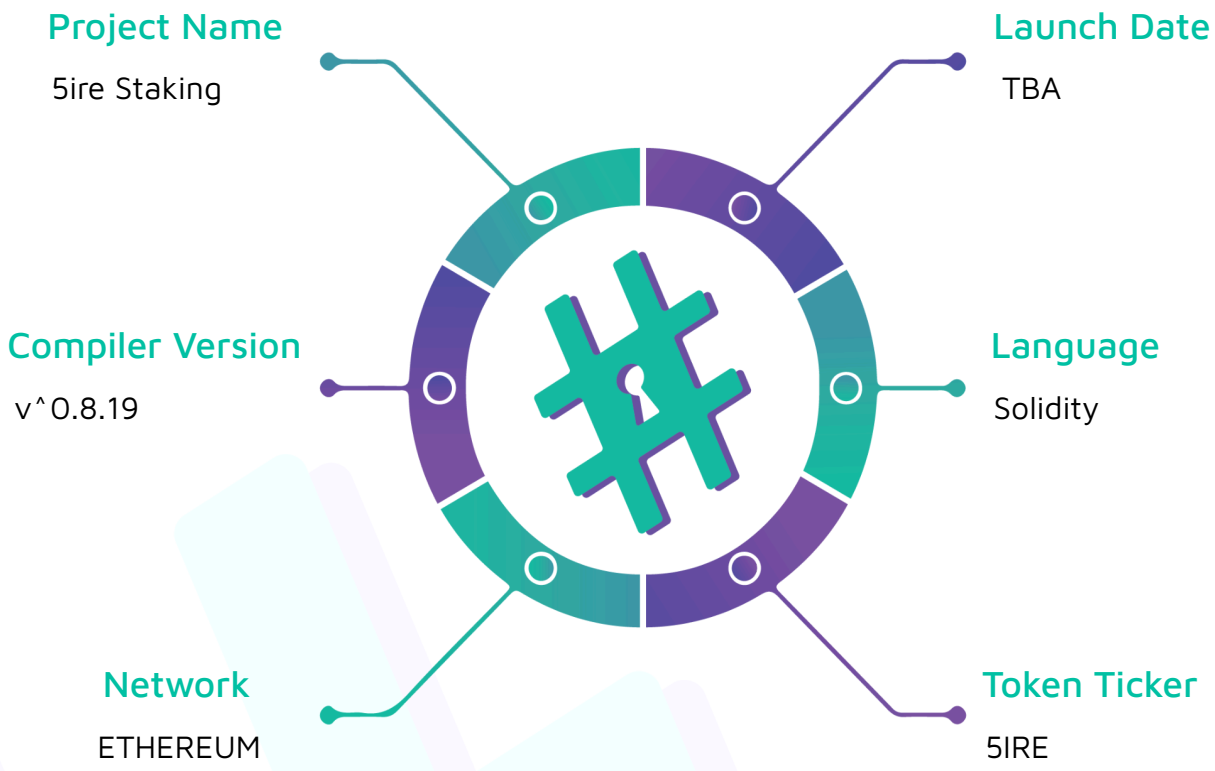
The 5ire team partnered with Hashlock to conduct a security audit of their IFireToken.sol, proxy.sol and Staking2.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

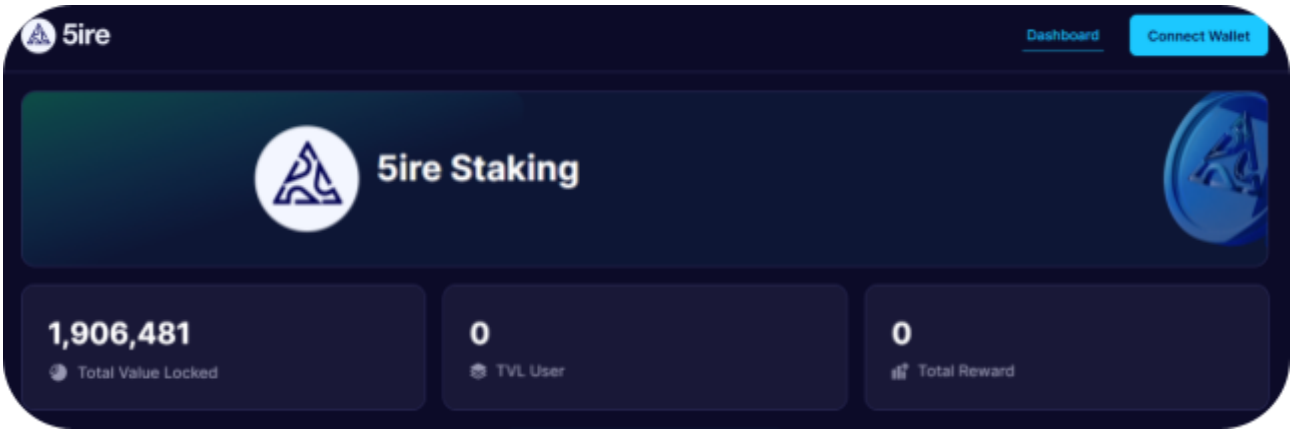
5ire is a DeFi dApp built on Ethereum Network that allows you to stake your 5ire tokens and earn 5ire token rewards.

Project Name: 5ire
Compiler Version: ^0.8.7, ^0.8.19
Website: https://staking.5ire.network/
Logo:

Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the 5ire project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	5ire Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2024
Contract 1	IFireChain.sol
Contract 1 MD5 Hash	f027ad9dff1ddc4dc50f36b9ee9a0ef1
Contract 2	proxy.sol
Contract 2 MD5 Hash	f1dba7f90d696316074cc1cf3bd4634c
Contract 3	Staking2.sol
Contract 3 MD5 Hash	4f59cee5686f54ec71e418dbef5b8f31

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

0 Medium severity vulnerabilities

2 Low severity vulnerabilities

4 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
proxy.sol - Proxy contract	**Contract achieves this functionality.**

Staking2.sol
- Allows users to:
  - Stake their 5ire tokens into different pools.
  - Earn 5ire token rewards
  - Stake their earned rewards.
  - Withdraw all their tokens and rewards.
- Allows admins to:
  - Add new staking pools and update existing pools.
  - Pause/unpause the contract.
	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the 5ire project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the 5ire projects smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] Staking2.sol - Storage collision in updated contracts breaks functions

Description

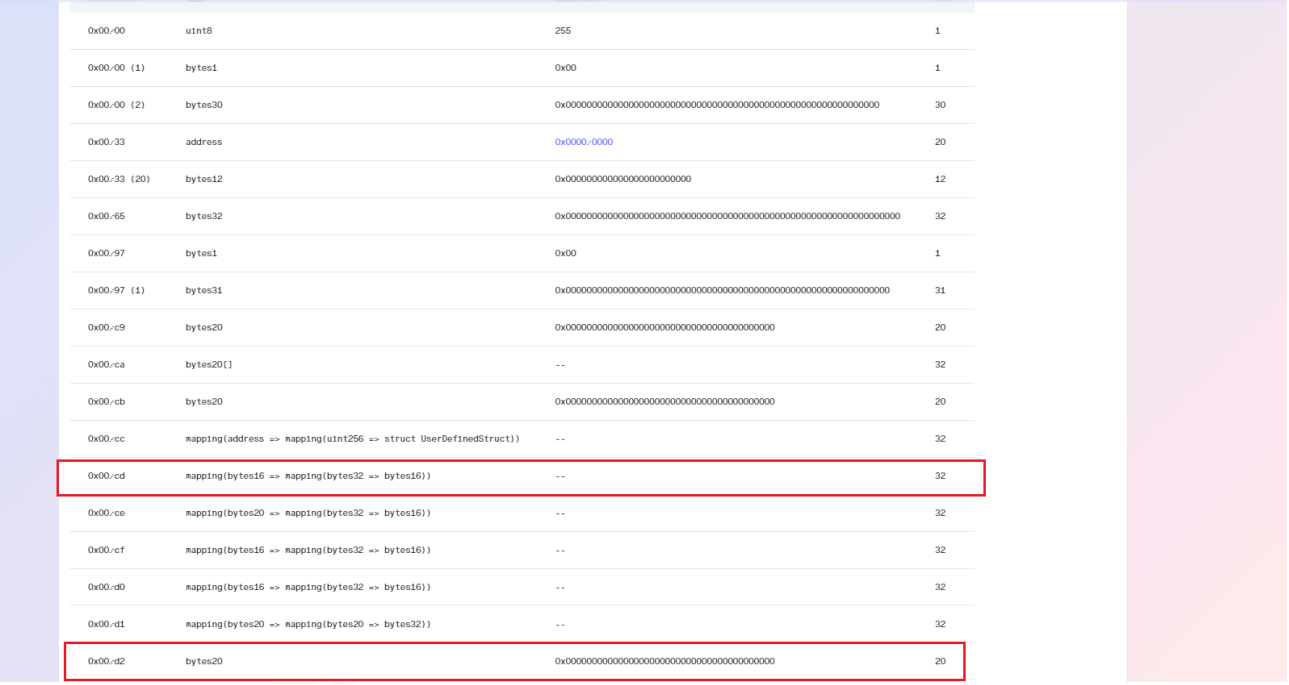
Storage collision in the updated contract will break the `addPool` function.

Vulnerability Details

Staking proxy is deployed at 0x83d1b4277454edacf54a14fc586326386648a959. It points to an implementation contract at 0x84eaf0a05febaf76aa6678d592223fbf85666705 but it used to point to 0x54ca3e74bd7843b6b6c40450a05af727c022d2c4:

Old implementation contract's storage layout can be checked at https://evm.storage/eth/19826296/0x54ca3e74bd7843b6b6c40450a05af727c022d2c4#table.

In the table view, notice the storage slot at `0xcd` stores a `mapping` and `0xd2` stores `bytes20` :

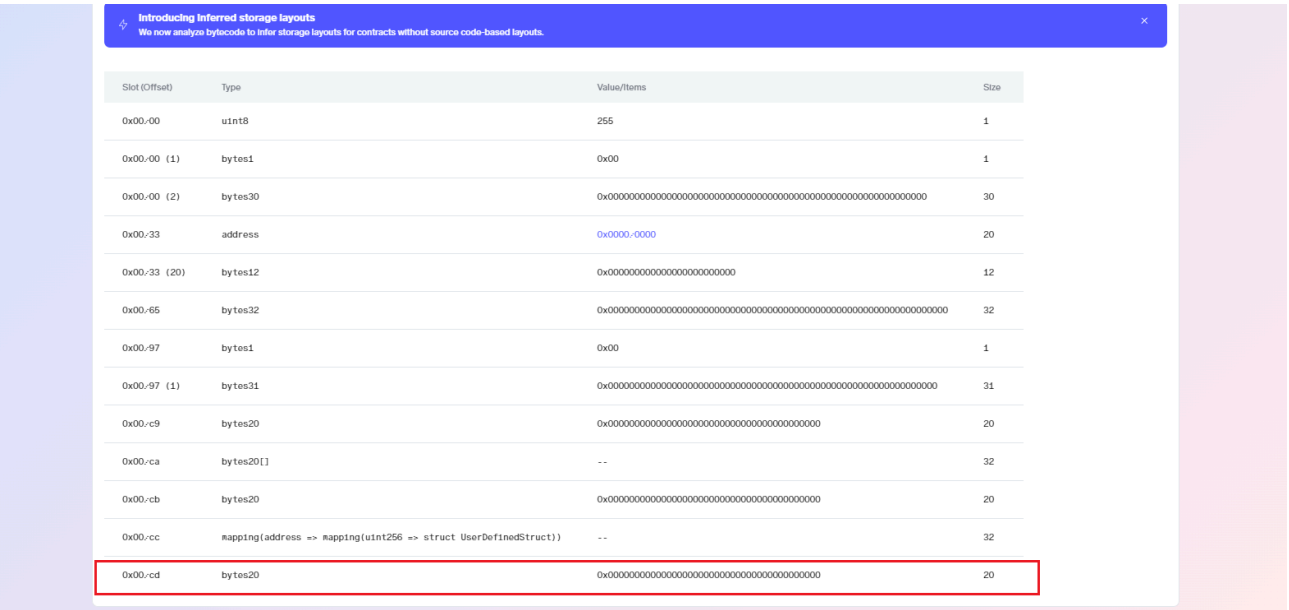


0x00	uint8	255	1
0x00 (1)	bytes1	0x00	1
0x00 (2)	bytes30	0x000000000000000000000000000000000000000000000000000000000000	30
0x00	address	0x0000000000000000000000000000000000000000	20
0x00 (20)	bytes12	0x000000000000000000000000	12
0x00	bytes32	0x0000000000000000000000000000000000000000000000000000000000000000	32
0x00	bytes1	0x00	1
0x00 (1)	bytes31	0x00000000000000000000000000000000000000000000000000000000000000	31
0x00	bytes20	0x0000000000000000000000000000000000000000	20
0x00	bytes20[]	--	32
0x00	bytes20	0x0000000000000000000000000000000000000000	20
0x00	mapping(address => mapping(uint256 => struct UserDefinedStruct))	--	32
0x00	mapping(bytes16 => mapping(bytes32 => bytes16))	--	32
0x00	mapping(bytes20 => mapping(bytes32 => bytes16))	--	32
0x00	mapping(bytes16 => mapping(bytes32 => bytes16))	--	32
0x00	mapping(bytes16 => mapping(bytes32 => bytes16))	--	32
0x00	mapping(bytes20 => mapping(bytes20 => bytes32))	--	32
0x00	bytes20	0x0000000000000000000000000000000000000000	20

By checking the source code, we can see that the value at `0xd2` is the `fireChainToken` variable.

Now, lets check the new implementation contract's storage layout at https://evm.storage/eth/19826295/0x84eaf0a05febaf76aa6678d592223fbf85666705#table.

Notice in the new layout `bytes20` points to `0xcd` which in the old layout was pointing to `mapping`. So the value of the `fireChainToken` variable is now 0 because the storage layout points to an empty storage slot.



Impact

`addPool` function will not work, resulting in protocol not being able to add new staking pools.

Recommendation

Add the removed storage variable back in the implementation contract. Upgrade the proxy with the new implementation contract.

Status

Resolved. The storage collusion was resolved with upgrade to the contract with address 0x6931d688bfc342cF25d383A93Df9f9dfE4116ca8. Subsequent upgrades didn't modify the storage layout.

[H-02] Staking2.sol::stakeFireTokenReward - Lack of checks leads to user staking and locking tokens after pool has ended

Description

The `stakeFireTokenReward` function lacks adequate checks to make sure that the staking pool has not ended when the user is staking their reward tokens.

Vulnerability Details

The `stakeFireTokenReward` function allows users to stake their earned rewards (interests) into the same pool in order to earn more. However, it doesn't check whether the pool has ended or will end while user funds are still locked.
If we take a look at the `stake` function:

```
function stake(
        uint256 _poolIndex,
        uint128 _amount
    ... //redacted for brevity
       if ((currentDay + _pool.lockPeriod) > _pool.endDay) {
            revert TooLate();

    ... //redacted for brevity
}
```

The shown "if" check verifies whether the pool will remain active until the staked tokens are unlocked. This implementation is crucial because once the staking pool ends, users won't earn any rewards for their staked tokens anymore.

We can confirm this by observing that earned rewards are calculated using the results from the `getMultiplier` function.

```
if (_userInfo.userDeposit != 0) {
    (uint128 lockDays, uint128 unlockDays) = getMultiplier(_userInfo.lastDayActionSec,
_pool.endDay, _pool.lockPeriod);
```

```
    uint128    lockInterest    =    _userInfo.boost    ?    ((_userInfo.userDeposit    *
_pool.boostDayPercent * lockDays) /  (PERCENT_BASE * 86400 * 365))

 : ((_userInfo.userDeposit * _pool.lockDayPercent * lockDays) / (PERCENT_BASE * 86400
* 365));

    uint128  unlockInterest  =  ((_userInfo.userDeposit  *  _pool.unlockDayPercent  *
unlockDays) / (PERCENT_BASE * 86400 * 365));

     _userInfo.accrueInterest += lockInterest + unlockInterest;

        }
```

Upon examining the `getMultiplier` function, we learn that it returns the duration for which a user's token has been locked and unlocked (available for withdrawal).

```
function getMultiplier(

        uint128 _lastDayAction,

        uint128 _poolEndDay,

        uint128 _poolLockPeriod

    ) internal view returns (uint128 lockDays, uint128 unlockDays) {

...

     if ((currentDay >= _lastDayAction) && (currentDay <= _poolEndDay)) {

     …

     } else {

          lockDays = 0;

          unlockDays = 0;

      }
```

Examining the "if" check above, we notice that if the `currentDay` is greater than `_poolEndDay`, the function won't proceed into the "if" check and will return 0, 0. This implies that if the staking pool has concluded, users' reward multipliers will be set to 0, and they won't be able to earn any more interest.

However, as the `stakeFireTokenReward` function doesn't verify if the pool has ended, users can stake their reward tokens, causing them to be locked for the pool's lock duration inadvertently. Consequently, a user might unintentionally stake more tokens this way, and these tokens could remain locked for up to 365 days without earning any rewards or being withdrawable.

Impact

Users who unintentionally stake reward tokens in this manner after a pool has ended will find themselves unable to withdraw their tokens, despite the fact that their tokens are no longer generating any rewards.

Recommendation

Implement the same check that the `stake` function has in the `stakeFireRewardToken` function to verify if the staking pool has ended.

```
        if ((currentDay + _pool.lockPeriod) > _pool.endDay) {

            revert TooLate();
```

Status

Resolved. Users are allowed to restake their reward tokens only if they do so within the designated staking period.

Low

[L-01] Contracts - Use Ownable2step rather than Ownable

Description

Ownable2Step and Ownable2StepUpgradeable prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2Step or Ownable2StepUpgradeable from OpenZeppelin Contracts to enhance the security of your contract ownership management. These contracts prevent the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call. This two-step ownership transfer process adds an additional layer of security to your contract's ownership management.

Status

Acknowledged.

[L-02] Staking2.sol - Centralization risk for privileged functions

Description

Contracts with privileged functions need the owner/admin to be trusted not to perform malicious updates. This may also cause a single point failure.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralised governance mechanism. Instead of relying solely on a single owner/administrator

Status

Acknowledged. Admin will use the Multi-sig wallet to access the privileged functionality.

[L-03] Staking2.sol::updatePool - Incorrect pool index validation

Description

`updatePool` function has a check that validates if the entered index argument is within array's existing pool range. However, this check is implemented incorrectly here:

```
        if (_poolIndex > pools.length) {

         revert IndexOutOfBound();
```

The above if statement should be greater than or equal to symbol. As otherwise, pool index can be equal to the pool's length which would lead to update of an non-existing pool.

Recommendation

Update the `updatePool` function as shown below:

```
        if (_poolIndex >= pools.length) {

         revert IndexOutOfBound();
```

Status

Resolved.

Gas

[G-01] Staking2.sol::getCurrentDay - Function can be removed.

Description

Function is used to get `uint128(block.timestamp)`, however this function is unnecessary as some functions in the contract make calls to this function while others do it manually.

Recommendation

Remove the function and change all the calls done to this function as seen in `getMultiplier` and `stakeFireTokenReward` functions.

Status

Resolved.

[G-02] Staking2.sol - Avoid repeating computations

Repeating the same computations within a contract can lead to unnecessary gas consumption

Recommendation

`(PERCENT_BASE * 86400 * 365))` computation is used multiple times in the contract, the result of this computation can be cached in a local variable instead to save gas.

Status

Resolved.

[G-03] Staking2.sol - State Variables That Are Used Multiple Times In a Function Should Be Cached In Stack Variables

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack. Caching of a state variable replaces each Gwarmaccess (100 gas) with a much cheaper stack read. Other less obvious fixes/optimizations include having local memory caches of state variable structs, or having local caches of state variable contracts/addresses. Saves 100 gas per instance.

Recommendation

State variables: `_userInfo` and `rewardWallet` are used multiple times. Caching these state variables in stack or local memory variables within functions when they are used multiple times will save gas.

Status

Resolved.

[G-04] Staking2.sol - Unnecessary reads to get token

Contract only uses the 5ire token, avoid making reads into structs and arrays to get the token.

Recommendation

Assign the 5ire token into a state variable to be used instead of `_pool.token`

Status

Acknowledged. The 5ire development team decided not to take any action, citing concerns about altering the storage structure of the already deployed mainnet contracts, potential risk of collision, and additional gas costs associated with introducing another state variable.

Centralisation

The Template project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the 5ire project seems to have a sound and well-tested code base, however, now that our findings have been resolved, or acknowledged, in order to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.